FIAP ON

COLHEITA DE SOLUÇÕES INTELIGENTES - TRANSFORMANDO DADOS EM AÇÕES

FUNÇÕES EM AÇÃO: APROFUNDANDO NO DRS

6

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Imagem das referências do estudo de caso 7
Figura 2 - Modelo de Dados Lógico 8
Figura 3 - Modelo de Dados Físico ou Relacional.....9
Figura 4 - Funções de caractere 10
Figura 5 - Resultado do SELECT com as funções LOWER(), UPPER() e INITCAP() 12
Figura 6 - Resultado do SELECT com as funções ASCII() e CHR() 13
Figura 7 - Resultado do SELECT com a função CONCAT() 14
Figura 8 - Resultado do SELECT, com a função INSTR() 15
Figura 9 - Resultado do SELECT com a função LENGTH ().....16
Figura 10 - Resultado do SELECT com as funções RPAD() e LPAD() 18
Figura 11 - Resultado do SELECT com as funções RTRIM(), LTRIM() e TRIM() 20
Figura 12 - Resultado do SELECT com as funções RTRIM(), LTRIM() E TRIM().....21
Figura 13 - Resultado do SELECT, com as funções NVL() e NVL2() 22
Figura 14 - Resultado do SELECT com a função REPLACE().....23
Figura 15 - Resultado do SELECT com a função SUBSTR ().....24
Figura 16 - Resultado do SELECT com as funções numéricas.....27
Figura 17 - Resultado do SELECT com as funções de conversão – TO_CHAR() 30
Figura 18 - Resultado do SELECT com as funções de conversão – TO_NUMBER() 31
Figura 19 - Resultado do SELECT, com as funções de conversão – TO_DATE()....31
Figura 20 - Resultado do SELECT com as funções de conversão – CAST().....33
Figura 21 - Resultado do SELECT com as funções de conversão – CAST().....34
Figura 22 - Resultado do SELECT com a função de data – SYSDATE 36
Figura 23 - Resultado do SELECT com as funções de data – ADD_MONTHS().....36
Figura 24 - Resultado do SELECT com as funções de data – ADD_MONTHS().....37
Figura 25 - Resultado do SELECT com as funções de data – LAST_DAY() 38
Figura 26 - Resultado do SELECT com as funções de data – LAST_DAY() 38
Figura 27 - Resultado do SELECT com as funções de data – MONTHS_BETWEEN() 39
Figura 28 - Resultado do SELECT com as funções de data – NEXT_DAY().....40
Figura 29 - Resultado do SELECT com as funções de data 42
Figura 30 - Resultado do SELECT com as funções de data 42
Figura 31 - Resultado do SELECT com as funções de data 43

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Funções de uma única linha 9
Quadro 2 - Funções de caractere – conversão de escrita.....11
Quadro 3 - Funções de caractere – manipulação de caracteres ASCII() e CHR() 12
Quadro 4 - Funções de caractere – manipulação de caracteres CONCAT() 13
Quadro 5 - Funções de caractere – manipulação de caracteres INSTR().....14
Quadro 6 - Funções de caractere – manipulação de caracteres LENGTH().....16
Quadro 7 - Funções de caractere – manipulação de caracteres RPAD() e LPAD()..17
Quadro 8 - Funções de caractere – manipulação de caracteres RTRIM(), LTRIM() e TRIM() 19
Quadro 9 - Funções de caractere – manipulação de caracteres NVL() e NVL2() 21
Quadro 10 - Funções de caractere – manipulação de caracteres REPLACE().....22
Quadro 11 - Funções de caractere – manipulação de caracteres SUBSTR().....23
Quadro 12 - Funções numéricas 26
Quadro 13 - Funções de conversão 28
Quadro 14 - Conversões válidas em um CAST() 32

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de código-fonte SQL – Funções LOWER(), UPPER(), INITCAP() 11
Código-fonte 2 - Exemplo de código-fonte SQL – Funções ASCII() e CHR() 12
Código-fonte 3 - Exemplo de código-fonte SQL – Função CONCAT() 13
Código-fonte 4 - Exemplo de código-fonte SQL – Função INSTR() 15
Código-fonte 5 - Exemplo de código-fonte SQL – Função LENGTH () 16
Código-fonte 6 - Exemplo de código-fonte SQL – Funções RPAD() e LPAD() 18
Código-fonte 7 - Exemplo de código-fonte SQL – Funções RTRIM(), LTRIM() E TRIM() 19
Código-fonte 8 - Exemplo de código-fonte SQL – Funções RTRIM(), LTRIM() E TRIM() 20
Código-fonte 9 - Exemplo de código-fonte SQL – Funções NVL() e NVL2() 22
Código-fonte 10 - Exemplo de código-fonte SQL – Função REPLACE() 23
Código-fonte 11 - Exemplo de código-fonte SQL – Função SUBSTR () 24
Código-fonte 12 - Exemplo de código-fonte SQL – Funções numéricas 27
Código-fonte 13 - Exemplo de código-fonte SQL – Funções de conversão – TO_CHAR() 29
Código-fonte 14 - Exemplo de código-fonte SQL – Funções de conversão – TO_NUMBER() 30
Código-fonte 15 - Exemplo de código-fonte SQL – Funções de conversão – TO_DATE() 31
Código-fonte 16 - Exemplo de código-fonte SQL – Funções de conversão – CAST() 33
Código-fonte 17 - Exemplo de código-fonte SQL – Funções de conversão – CAST() 34
Código-fonte 18 - Exemplo de código-fonte SQL – Funções de data – SYSDATE 35
Código-fonte 19 - Exemplo de código-fonte SQL – Funções de data – ADD_MONTHS() 36
Código-fonte 20 - Exemplo de código-fonte SQL – Funções de data – ADD_MONTHS() 37
Código-fonte 21 - Exemplo de código-fonte SQL – Funções de data – LAST_DAY() 37
Código-fonte 22 - Exemplo de código-fonte SQL – Funções de data – LAST_DAY() 38
Código-fonte 23 - Exemplo de código-fonte SQL – Funções de data – MONTHS_BETWEEN() 39
Código-fonte 24 - Exemplo de código-fonte SQL – Funções de data – NEXT_DAY() 40
Código-fonte 25 - Exemplo de código-fonte SQL – Funções de data 41
Código-fonte 26 - Exemplo de código-fonte SQL – Funções de data 42
Código-fonte 27 - Exemplo de código-fonte SQL – Funções de data 43

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 FUNÇÕES EM AÇÃO: APROFUNDANDO NO DRS 6
1.1 Estudo de caso utilizado para exemplificar e implementar os comandos DRS/DQL 6
1.2 Utilizando funções – Funções de uma única linha 9
1.3 Utilizando funções – Funções de caractere.....10
1.3.1 Função SQL – Conversão de escrita 10
1.3.2 Função SQL – Manipulação de caracteres – ASCII() e CHR().....12
1.3.3 Função SQL – Manipulação de caracteres – CONCAT().....13
1.3.4 Função SQL – Manipulação de caracteres – INSTR() 14
1.3.5 Função SQL – Manipulação de caracteres – LENGTH().....16
1.3.6 Função SQL – Manipulação de caracteres – LPAD() e RPAD() 17
1.3.7 Função SQL – Manipulação de caracteres – LTRIM(), RTRIM() e TRIM().....18
1.3.8 Função SQL – Manipulação de caracteres – NVL() e NVL2().....21
1.3.9 Função SQL – Manipulação de caracteres – REPLACE() 22
1.3.10 Função SQL – Manipulação de caracteres – SUBSTR().....23

2 UTILIZANDO FUNÇÕES – FUNÇÕES NUMÉRICAS.....25
2.1 Função SQL – Funções Numéricas 25

3 UTILIZANDO FUNÇÕES – FUNÇÕES DE CONVERSÃO 28
3.1 Função SQL – Funções de conversão 28
3.2 Função SQL – Exemplo: TO_CHAR().....29
3.3 Função SQL – Exemplo: TO_NUMBER().....30
3.4 Função SQL – Exemplo: TO_DATE() 31
3.5 Função SQL – Exemplo: CAST (X as TIPO) 32

4 UTILIZANDO FUNÇÕES – FUNÇÕES DE DATA.....35
4.1 Função SQL – Funções de data.....35
4.2 Função SQL – Exemplo: SYSDATE.....35
4.3 Função SQL – Exemplo: ADD_MONTHS() 36
4.4 Função SQL – Exemplo: LAST_DAY().....37
4.5 Função SQL – Exemplo: MONTHS_BETWEEN ().....38
4.6 Função SQL – Exemplo: NEXT_DAY () 39

5 EXEMPLOS ADICIONAIS ENVOLVENDO DATAS 41
5.1 Função SQL – Exemplo: TO_CHAR () 41
5.2 Função SQL – Exemplo: TO_DATE () 42

REFERÊNCIAS.....44

GLOSSÁRIO 45

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 FUNÇÕES EM AÇÃO: APROFUNDANDO NO DRS

Este capítulo irá tratar dos comandos DRS, agora é a vez das funções. Existem várias delas como: funções de data, conversão, manipulação de caracteres. Saiba para que servem e como utilizá-las, descubra tudo isso praticando e consumindo o conteúdo deste capítulo!

Na linguagem SQL temos instruções que nos permitem recuperar os dados de tabelas de um banco de dados. Essas instruções pertencem à divisão DRS (Data Retrieve Statement) ou DQL (Data Query Language) da linguagem SQL.

As funções não são necessariamente padronizadas, logo, podemos ter funções que existem no SGBD SQL Server e que não existem no SGBD Oracle.

As funções devem retornar sempre **um** valor, podendo, eventualmente, voltar **mais do que um**. Nesse caso, o seu uso em comandos SQL DML pode ter restrições.

As funções podem ter parâmetros (um ou mais) que passam informações para que a função possa ser processada. Porém, existem funções que não precisam de valores para serem processadas, no caso, existe apenas um retorno.

Os parâmetros de uma função normalmente são identificados por estarem dentro de parênteses, e ficam separados por vírgula (caso sejam maiores que um).

O retorno de uma função pode ser um valor numérico, "string", data ou, eventualmente, NULO.

1.1 Estudo de caso utilizado para exemplificar e implementar os comandos DRS/DQL

- **Estudo de Caso: SIP – Sistema de Controle de Implantação de Projetos**

A seguir, considere a imagem que representa um conjunto de funcionários da empresa XYZ, responsável pela implantação de projetos desta empresa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

FUNCIONARIO	MATRÍCULA	DATA NASC.	DATA ADMISSÃO	DEPARTAMENTO	SALÁRIO	ENDEREÇO
Ana Maria	1234	10/02/1986	09/08/2010	01 - Financeiro	R$ 1.234,56	R. Dario Ferreira, 23
Rosa da Silva	1235	15/08/1970	07/06/2005	02 - Marketing	R$ 2.345,00	Trav. Das Lágrimas, 34
Antonia Camargo	3245	18/07/1986	11/02/2003	01 - Financeiro	R$ 7.654,99	Pç. XV de Novembro, 219
João da Silva	2233		08/09/2011	03 - Recursos Humanos	R$ 3.452,12	Av. Nações Unidas, 10200
Joaquim Xavier	7866	21/04/1987	05/04/2010	03 - Recursos Humanos	R$ 980,00	R. Gil Vicente, 45
José Maria	9876	08/09/1998	17/03/2009	04 - Comercial	R$ 1.780,00	Av. Paulista, 23

FUNCIONARIO	DEPENDENTE	DATA NASC.
Rosa da Silva	Pedro da Silva	12/01/1983
Antonia Camargo	George Camargo	15/09/1971
Antonia Camargo	Antonio Camargo	10/06/2000
João da Silva	Ana da Silva	23/04/1976
João da Silva	Joana da Silva	16/05/2002
João da Silva	João da Silva Jr.	16/05/2002

PROJETO	DATA INÍCIO	DATA TÉRMINO
001 - PROJETO ABC	10/01/2013	
002 PROJETO BCD	28/10/2012	10/03/2013
003 - PROJETO CDE	09/12/2012	
004 - PROJETO DEF	15/12/2012	

PROJETO	FUNCIONÁRIO	DATA ENTRADA	DATA SAÍDA
001 - PROJETO ABC	Antonia Camargo	10/01/2013	
002 PROJETO BCD	Antonia Camargo	28/10/2012	10/03/2013
003 - PROJETO CDE	João da Silva	09/12/2012	
004 - PROJETO CDE	Joaquim Xavier	15/01/2013	

Figura 1 - Imagem das referências do estudo de caso
Fonte: Elaborado pela autora (2017)

Regras de Negócio:

- RN01 – Um funcionário tem apenas um endereço residencial.
- RN02 – Um funcionário pode ter um ou mais dependentes (filhos, esposa ou marido).
- RN03 – Um funcionário tem apenas uma data de admissão.
- RN04 – Um funcionário deve pertencer a um único departamento.
- RN05 – Um funcionário tem um único número de matrícula.
- RN06 - Um funcionário pode participar da implantação de um ou mais projetos.
- RN07 – Um departamento pode locar vários funcionários.
- RN08 – Um projeto pode ser implantado por vários funcionários.
- RN09 – Cada dependente deve ter uma data de nascimento.
- RN10 – Cada dependente pertence a um único funcionário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- RN11 – Um funcionário pode participar da implantação de um mesmo projeto várias vezes, ou seja, um funcionário pode participar da implantação de um projeto em diferentes momentos da implantação.
- RN12 – Não será considerada a hipótese de termos um casal trabalhando na mesma empresa (teriam dependentes comuns).

Modelo de Dados Lógico:

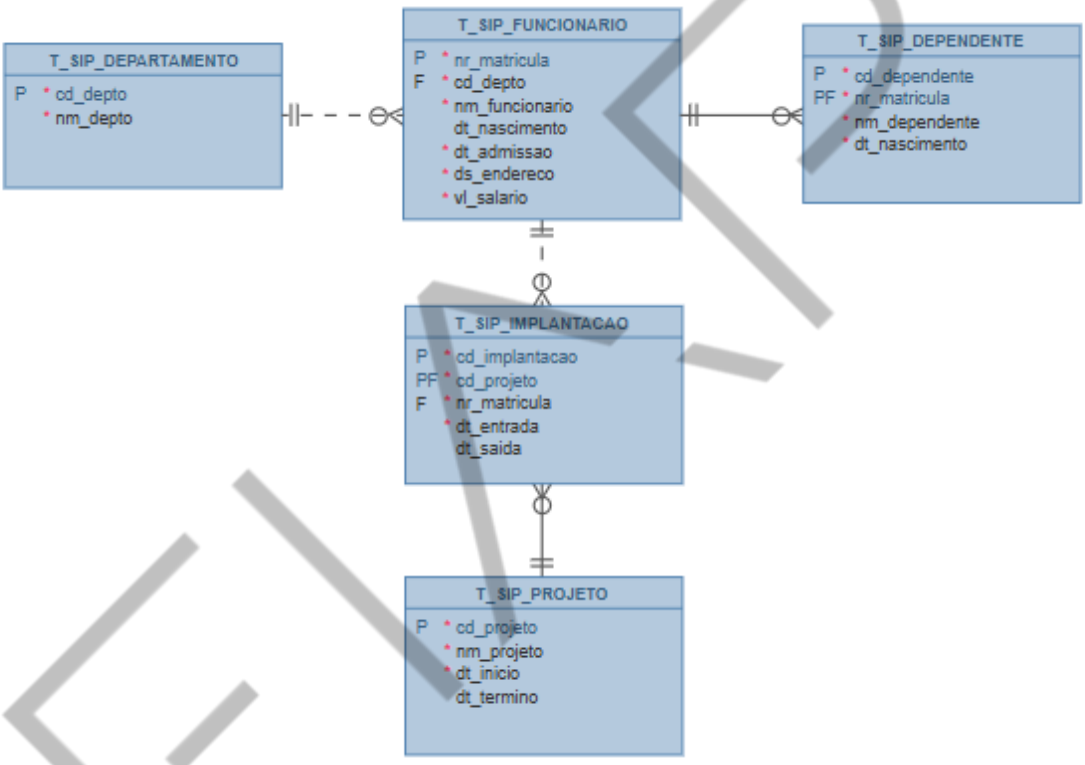

Figura 2 - Modelo de Dados Lógico
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SQL Developer Data Modeler: Modelo de Dados Físico ou Relacional:

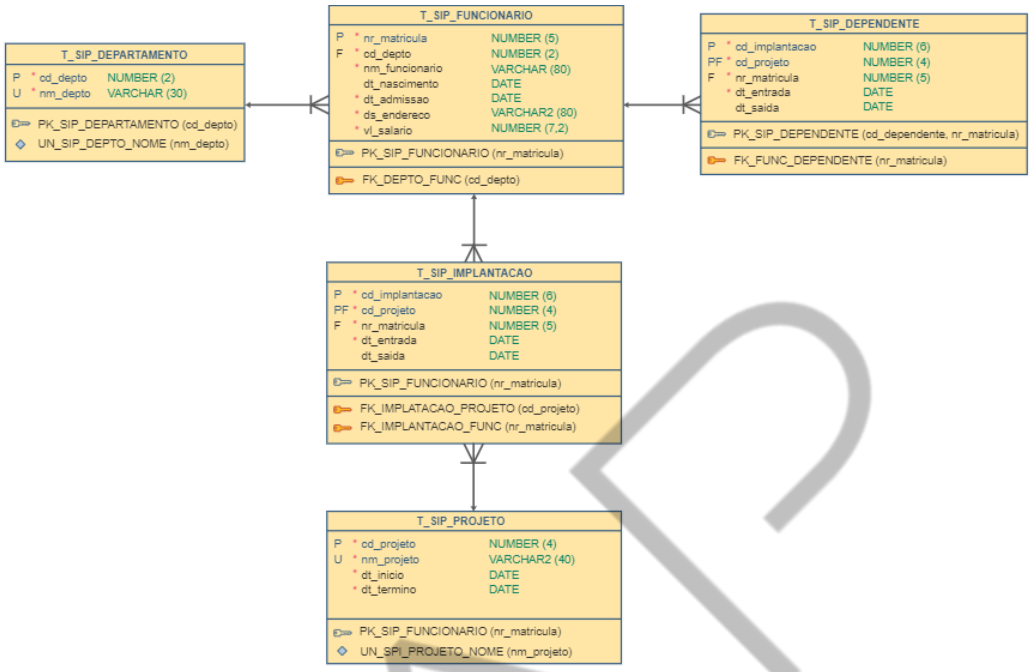

Figura 3 - Modelo de Dados Físico ou Relacional
Fonte: Elaborado pela autora (2017)

1.2 Utilizando funções – Funções de uma única linha

Funções de uma única linha: opera sobre uma linha por vez e retorna uma linha de saída para cada linha.

Existem cinco tipos de funções de uma única linha, das quais abordaremos as quatro primeiras:

Função	Descrição
CARACTERE	Manipulam strings de caractere.
NÚMERICAS	Efetuam cálculos.
CONVERSÃO	Convertem um valor de um tipo de banco de dados para outro.
DATA	Processam datas e horas.
EXPRESSÃO REGULAR	Utilizam expressões regulares para procurar dados, introduzidas no Oracle 10g e ampliadas no 11g.

Quadro 1 - Funções de uma única linha
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.3 Utilizando funções – Funções de caractere

São funções utilizadas para manipulação de strings de caracteres. As funções abordadas seguem a sintaxe padronizada pela Oracle.

As funções de caracteres são divididas em: funções para conversão de escrita (para manipulação de maiúsculos e minúsculos) e funções para manipulação de cadeia de caracteres, como mostra a figura a seguir.

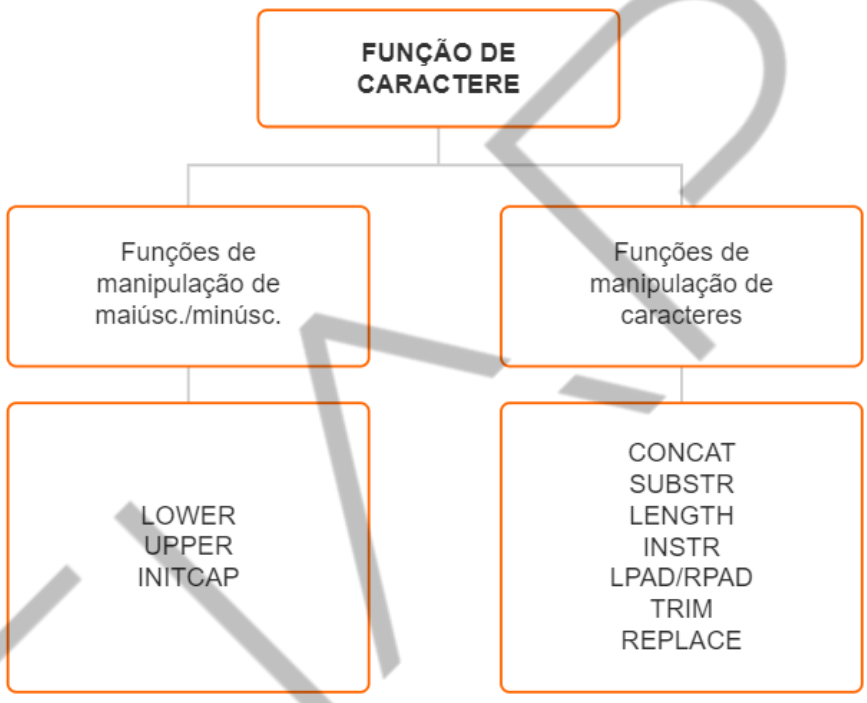

Figura 4 - Funções de caractere
Fonte: Price (2009)

1.3.1 Função SQL – Conversão de escrita

São funções para manipulação de caracteres, por meio das quais convertermos as letras informadas para caracteres maiúsculos ou minúsculos. A função retornará uma nova *string* como resultado da função.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Função	Descrição
LOWER(**x**)	Converte as letras de **x** para minúsculas e retorna a nova string.
UPPER(**x**)	Converte as letras de **x** para maiúsculas e retorna a nova string.
INITCAP(**x**)	Converte a letra inicial de cada palavra da string **x** em maiúsculas e retorna a nova string.

Quadro 2 - Funções de caractere – conversão de escrita
Fonte: Elaborado pela autora (2017)

O comando SELECT, através das funções LOWER(), UPPER() e INITCAP() retornará, respectivamente, as letras informadas na cadeia de caracteres, em letras minúsculas, maiúsculas e a letra inicial de cada palavra em letra maiúscula.

O conteúdo, conforme está armazenado na tabela, também é apresentado para que possa ser verificada a aplicação de cada uma das funções.

```
      -- EXEMPLO – FUNÇÕES LOWER(), UPPER() e INITCAP()
SELECT F.NM_FUNCIONARIO            "COMO ESTÁ GRAVADO"   ,
       LOWER(F.NM_FUNCIONARIO)     "MINUSCULAS (LOWER)" ,
       UPPER(F.NM_FUNCIONARIO)     "MAIUSCULAS (UPPER)" ,
       INITCAP(F.NM_FUNCIONARIO)  "MAIUSCULA/MINUSCULA (INITCAP)"
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 1 - Exemplo de código-fonte SQL – Funções LOWER(), UPPER(), INITCAP()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções: LOWER(), UPPER() e INITCAP().

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

COMO ESTÁ GRAVADO	MINUSCULAS (LOWER)	MAISCULAS (UPPER)	MAIUSCULA/MINUSCULA (INITCAP)
ANA MARIA	ana maria	ANA MARIA	Ana Maria
ROSA MARIA	rosa maria	ROSA MARIA	Rosa Maria
ANTONIA CAMARGO	antonia camargo	ANTONIA CAMARGO	Antonia Camargo
JOÃO DA SILVA	joão da silva	JOÃO DA SILVA	João da Silva
JOAQUIM XAVIER	joaquim xavier	JOAQUIM XAVIER	Joaquim Xavier
JOSÉ MARIA	josé maria	JOSÉ MARIA	José Maria
JOSÉ LISBOA	josé lisboa	JOSÉ LISBOA	José Lisboa

Figura 5 - Resultado do SELECT com as funções LOWER(), UPPER() e INITCAP()
Fonte: Elaborado pela autora (2017)

1.3.2 Função SQL – Manipulação de caracteres – ASCII() e CHR()

São funções para manipulação de caracteres, por meio das quais é possível retornar o código ASCII ou retornar o caractere correspondente ao código ASCII informado.

Função	Descrição
ASCII(**x**)	Retorna o código ASCII do caractere **x**.
CHR(**x**)	Retorna o caractere referente ao código ASCII informado em **x**.

Quadro 3 - Funções de caractere – manipulação de caracteres ASCII() e CHR()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através das funções ASCII() e CHR(), retornará, respectivamente, o código ASCII da letra ‘R’ e o caractere correspondente ao código ASCII 82. Apontamos para a tabela DUAL (tabela do Oracle), pois o conteúdo não está sendo recuperado de nenhuma tabela do modelo.

```
-- EXEMPLO – FUNÇÕES ASCII() e CHR()
SELECT ASCII('R'),
       CHR(82)
  FROM DUAL;
```

Código-fonte 2 - Exemplo de código-fonte SQL – Funções ASCII() e CHR()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução do comando “SELECT” com as funções ASCII() e CHR().

ASCII('R')	CHR(82)
82	R

Figura 6 - Resultado do SELECT com as funções ASCII() e CHR()
Fonte: Elaborado pela autora (2017)

1.3.3 Função SQL – Manipulação de caracteres – CONCAT()

É uma função para manipulação de caracteres, por meio da qual é possível concatenar (unir - ligar) valores. A função retornará o valor concatenado em uma nova string.

Função	Descrição
CONCAT(**x,y**)	Anexa o conteúdo de **y** e **x** e retorna a nova string.

Quadro 4 - Funções de caractere – manipulação de caracteres CONCAT()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através da função CONCAT(), retornará primeiramente a execução da função CONCAT() no parênteses mais interno, em que será concatenado o número de matrícula do funcionário ao hífen. Após, será executada a função CONCAT() referente ao parênteses externo, que concatenará a *string* retornada (matrícula do funcionário mais hífen) ao nome do funcionário, resultando em número de matrícula do funcionário mais hífen, mais o nome do funcionário.

```
      -- EXEMPLO – FUNÇÃO CONCAT()
SELECT CONCAT(CONCAT(F.NR_MATRICULA,'-'),F.NM_FUNCIONARIO)
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 3 - Exemplo de código-fonte SQL – Função CONCAT()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função CONCAT().

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCAT(CONCAT(F.NR_MATRICULA,'-'),F.NM_FUNCIONARIO)
1234-ANA MARIA
1235-ROSA MARIA
3245-ANTONIA CAMARGO
2233-JOÃO DA SILVA
7866-JOAQUIM XAVIER
9876-JOSÉ MARIA
9899-JOSÉ LISBOA

Figura 7 - Resultado do SELECT com a função CONCAT()
Fonte: Elaborado pela autora (2017)

1.3.4 Função SQL – Manipulação de caracteres – INSTR()

É uma função para manipulação de caracteres, por meio da qual é possível localizar um caractere ou cadeia de caracteres em qualquer parte de uma *string*. A função retornará à posição em que ocorre o caractere ou cadeia de caracteres informado.

Função	Descrição
INSTR(**x, LOCALIZAR_STRING [,INÍCIO] [,OCORRÊNCIA]**)	Procura "**localizar_String**" em **x** e retorna a posição em que ocorre. Pode ser informada a posição **início** (opcional) para iniciar a busca. Pode ser fornecida a **ocorrência**(opcional), que indica qual **ocorrência** de "**localizar_String**" deve ser retornada.

Quadro 5 - Funções de caractere – manipulação de caracteres INSTR()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através da função INSTR(), retornará a posição do caractere especificado para localização dentro de uma *string*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- No exemplo **INSTR(**F.NM_FUNCIONARIO,'A'**):** será retornada a posição da primeira ocorrência encontrada para a letra 'A', a partir da *string* armazenada no campo nome do funcionário.
- No exemplo **INSTR(**F.NM_FUNCIONARIO,'JOS'**):** será retornada a posição da primeira ocorrência encontrada para a cadeia de caracteres 'JOS', a partir da *string* armazenada no campo nome do funcionário.
- No exemplo **INSTR(**F.NM_FUNCIONARIO,'A',3**):** será retornada a posição da primeira ocorrência encontrada para a letra 'A', a partir da terceira posição da *string*, utilizando a *string* armazenada no campo nome do funcionário para localização.
- No exemplo **INSTR(**F.NM_FUNCIONARIO,'A',3,2**):** será retornada a posição da segunda ocorrência encontrada para a letra 'A', a partir da terceira posição da *string*, utilizando a string armazenada no campo nome do funcionário para localização.

```
      -- EXEMPLO - FUNÇÃO INSTR()
SELECT F.NM_FUNCIONARIO                 ,
       INSTR(F.NM_FUNCIONARIO,'A')      ,
       INSTR(F.NM_FUNCIONARIO,'JOS')    ,
       INSTR(F.NM_FUNCIONARIO,'A',3)    ,
       INSTR(F.NM_FUNCIONARIO,'A',3,2)
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 4 - Exemplo de código-fonte SQL – Função INSTR()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando "SELECT" com a função INSTR().

NM_FUNCIONARIO	INSTR(F.NM_FUNCIONARIO,'A')	INSTR(F.NM_FUNCIONARIO,'JOS')	INSTR(F.NM_FUNCIONARIO,'A',3)	INSTR(F.NM_FUNCIONARIO,'A',3,2
ANA MARIA	1	0	3	6
ROSA MARIA	4	0	4	7
ANTONIA CAMARGO	1	0	7	10
JOÃO DA SILVA	7	0	7	13
JOAQUIM XAVIER	3	0	3	10
JOSÉ MARIA	7	1	7	10
JOSÉ LISBOA	11	1	11	0

Figura 8 - Resultado do SELECT, com a função INSTR()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.3.5 Função SQL – Manipulação de caracteres – LENGTH()

É uma função para manipulação de caracteres, e através dela será possível recuperar o tamanho em número de caracteres de uma *string*.

Função	Descrição
LENGTH(**x**)	Retorna o tamanho em caracteres do **x**.

Quadro 6 - Funções de caractere – manipulação de caracteres LENGTH()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através da função LENGTH(), retornará o tamanho, em número de caracteres, da *string* informada. No exemplo, será recuperado o tamanho em número de caracteres de cada nome armazenado no campo nome do funcionário, incluindo caracteres especiais, como espaço em branco, hífen, ponto ou qualquer outro caractere.

```
      -- EXEMPLO – FUNÇÃO LENGTH ()
SELECT F.NM_FUNCIONARIO ,
       LENGTH(F.NM_FUNCIONARIO)
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 5 - Exemplo de código-fonte SQL – Função LENGTH ()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função LENGTH ().

NM_FUNCIONARIO	LENGTH(F.NM_FUNCIONARIO)
ANA MARIA	9
ROSA MARIA	10
ANTONIA CAMARGO	15
JOÃO DA SILVA	13
JOAQUIM XAVIER	14
JOSÉ MARIA	10
JOSÉ LISBOA	11

Figura 9 - Resultado do SELECT com a função LENGTH ()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.3.6 Função SQL – Manipulação de caracteres – LPAD() e RPAD()

São funções utilizadas para realizar a manipulação de caracteres, e através delas é possível preencher uma cadeia de caracteres com espaços (padrão) ou outro caractere (a ser definido), à direita ou à esquerda, para que a *string* atinja n caracteres da largura informada.

Função	Descrição
LPAD(x, largura [,string_preencimento])	Preenche **x** com espaços a esquerda para que o comprimento total da string tenha até n caracteres de **largura**. Pode ser fornecida uma string para o preenchimento a esquerda de **x**. Quando não fornecida a string de preenchimento é utilizado o padrão espaços em branco.
RPAD(x, largura [,string_preenchimento])	Preenche **x** com espaços a direita para que o comprimento total da string tenha até n caracteres de **largura**. Pode ser fornecida uma string para o preenchimento a direita de **x**. Quando não fornecida a string de preenchimento é utilizado o padrão espaços em branco.

Quadro 7 - Funções de caractere – manipulação de caracteres RPAD() e LPAD()
Fonte: Elaborado pela autora (2017)

O comando SELECT, por meio das funções RPAD() e LPAD(), preencherá uma *string* à direita ou à esquerda com espaços em branco (padrão) ou com um outro caractere (a ser definido), até atingir a largura informada, através do parâmetro da função.

- No exemplo **RPAD(**F.NM_FUNCIONARIO,60**):** será preenchido com espaços em branco à direita do nome do funcionário, até que atinja a largura de 60 caracteres.
- No exemplo **RPAD(**F.NM_FUNCIONARIO,60,'.'**):** será preenchido com '.' (ponto) à direita do nome do funcionário, até que atinja a largura de 60 caracteres.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- No exemplo **LPAD(**F.VL_SALARIO,10,'*'**):** será preenchido com '*' (asterisco) à esquerda do salário do funcionário, até que atinja a largura de 10 caracteres.

```
-- EXEMPLO – FUNÇÕES RPAD() e LPAD()
SELECT F.NM_FUNCIONARIO                    ,
       F.VL_SALARIO                        ,
       RPAD(F.NM_FUNCIONARIO,60)           ,
       RPAD(F.NM_FUNCIONARIO,60,'.')       ,
       LPAD(F.VL_SALARIO,10,'*')
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 6 - Exemplo de código-fonte SQL – Funções RPAD() e LPAD()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando "SELECT" com as funções RPAD() e LPAD().

NM_FUNCIONARIO	VL_SALARIO	RPAD(F.NM_FUNCIONARIO,60)	RPAD(F.NM_FUNCIONARIO,60,'.')	LPAD(VL_SALARIO,10,'*')
ANA MARIA	1234,56	ANA MARIA ...	ANA MARIA.....	***1234,56
ROSA MARIA	2345	ROSA MARIA ...	ROSA MARIA.....	******2345
ANTONIA CAMARGO	7654,99	ANTONIA CAMARGO ...	ANTONIA CAMARGO.....	***7654,99
JOÃO DA SILVA	3452,12	JOÃO DA SILVA ...	JOÃO DA SILVA.....	***3452,12
JOAQUIM XAVIER	980	JOAQUIM XAVIER ...	JOAQUIM XAVIER.....	*******980
JOSÉ MARIA	1789	JOSÉ MARIA ...	JOSÉ MARIA.....	******1789
JOSÉ LISBOA	1780	JOSÉ LISBOA ...	JOSÉ LISBOA.....	******1780

Figura 10 - Resultado do SELECT com as funções RPAD() e LPAD()
Fonte: Elaborado pela autora (2017)

1.3.7 Função SQL – Manipulação de caracteres – LTRIM(), RTRIM() e TRIM()

São funções utilizadas para realizar a manipulação de caracteres, tornando possível remover espaços (padrão) ou outro caractere (a ser definido), à direita, esquerda e à direita e esquerda da cadeia de caracteres informada.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Função	Descrição
LTRIM(**x[,string_corte]**)	Corta caracteres a esquerda de **x**. **String_corte** (opcional) específica os caracteres a serem cortados. Se nenhuma string for fornecida por padrão são cortados os espaços em branco.
RTRIM(**x**[,string_corte])	Corta caracteres a direita de **x**. **String_corte** (opcional) específica os caracteres a serem cortados. Se nenhuma string for fornecida por padrão são cortados os espaços em branco.
TRIM(**x**[car_corte FROM]**x**)	Corta caracteres a direita e a esquerda de **x**. **String_corte** (opcional) específica os caracteres a serem cortados. Se nenhuma string for fornecida por padrão são cortados os espaços em branco.

Quadro 8 - Funções de caractere – manipulação de caracteres RTRIM(), LTRIM() e TRIM()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através das funções RTRIM(), LTRIM() e TRIM(), removerão espaços em branco (padrão) ou outro caractere (a ser definido), à direita, à esquerda e à direita e esquerda da cadeia de caracteres informada.

- No exemplo **LTRIM(**D.NM_DEPTO**):** serão removidos os espaços em branco à esquerda do campo nome do departamento, caso existam.
- No exemplo **TRIM(**D.NM_DEPTO**):** serão removidos os espaços em branco à direita e à esquerda do campo nome do departamento, caso existam.

```
-- EXEMPLO – FUNÇÕES RTRIM(), LTRIM() E TRIM()
SELECT D.NM_DEPTO          ,
       LTRIM(D.NM_DEPTO)   ,
       TRIM(D.NM_DEPTO)
  FROM T_SIP_DEPARTAMENTO D;
```

Código-fonte 7 - Exemplo de código-fonte SQL – Funções RTRIM(), LTRIM() E TRIM()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução do comando “SELECT” com as funções RTRIM(), LTRIM() e TRIM().

NM_DEPTO	LTRIM(D.NIM_DEPTO)	TRIM(D.NM_DEPTO)
CONTAS A PAGAR	CONTAS A PAGAR	CONTAS A PAGAR
COMERCIAL	COMERCIAL	COMERCIAL
FINANCEIRO	FINANCEIRO	FINANCEIRO
MARKETING	MARKETING	MARKETING
RECURSOS HUMANOS	RECURSOS HUMANOS	RECURSOS HUMANOS
TECNOLOGIA DA INFORMAÇÃO	TECNOLOGIA DA INFORMAÇÃO	TECNOLOGIA DA INFORMAÇÃO

Figura 11 - Resultado do SELECT com as funções RTRIM(), LTRIM() e TRIM()
Fonte: Elaborado pela autora (2017)

O comando SELECT, por meio das funções RTRIM(), LTRIM() e TRIM(), preencherá uma *string* à direita ou à esquerda com espaços em branco (padrão) ou com um outro caractere (a ser definido), até atingir a largura informada, através do parâmetro da função.

- No exemplo **LTRIM(**F.NM_FUNCIONARIO, 'A'**):** será removido o caractere ‘A’ à esquerda do campo nome do funcionário, caso exista.
- No exemplo **RTRIM(**F.NM_FUNCIONARIO, 'O'**):** será removido o caractere ‘O’ à direita do campo nome do funcionário, caso exista.
- No exemplo **TRIM('A' FROM** F.NM_FUNCIONARIO**):** será removido o caractere ‘A’ à direita e à esquerda do campo nome do funcionário, caso exista.

```
-- EXEMPLO – FUNÇÕES RTRIM(), LTRIM() E TRIM()
SELECT F.NM_FUNCIONARIO                    ,
       LTRIM(F.NM_FUNCIONARIO, 'A')        ,
       RTRIM(F.NM_FUNCIONARIO, 'O')        ,
       TRIM('A' FROM F.NM_FUNCIONARIO)
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 8 - Exemplo de código-fonte SQL – Funções RTRIM(), LTRIM() E TRIM()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução do comando “SELECT” com as funções RTRIM(), LTRIM() e TRIM().

NM_FUNCIONARIO	LTRIM(F.NM_FUNCIONARIO,'A')	RTRIM(F.NM_FUNCIONARIO,'O')	TRIM('A'FROMF.NM_FUNCIONARIO)
ANA MARIA	NA MARIA	ANA MARIA	NA MARI
ROSA MARIA	ROSA MARIA	ROSA MARIA	ROSA MARI
ANTONIA CAMARGO	NTONIA CAMARGO	ANTONIA CAMARGO	NTONIA CAMARGO
JOÃO DA SILVA	JOÃO DA SILVA	JOÃO DA SILVA	JOÃO DA SILV
JOAQUIM XAVIER	JOAQUIM XAVIER	JOAQUIM XAVIER	JOAQUIM XAVIER
JOSÉ MARIA	JOSÉ MARIA	JOSÉ MARIA	JOSÉ MARI
JOSÉ LISBOA	JOSÉ LISBOA	JOSÉ LISBOA	JOSÉ LISBO

Figura 12 - Resultado do SELECT com as funções RTRIM(), LTRIM() E TRIM()
Fonte: Elaborado pela autora (2017)

1.3.8 Função SQL – Manipulação de caracteres – NVL() e NVL2()

São funções para manipulação de caracteres, por meio das quais é possível retornar um valor predeterminado, caso o valor da cadeia de caracteres seja nulo, e o próprio valor, caso a cadeia de caracteres seja não nula, e, ainda, valores predeterminados, caso o valor da cadeia de caracteres seja não nulo ou nulo.

Função	Descrição
NVL(**x, valor**)	Retorna **valor**, caso **x** seja nulo; caso contrário, **x** será retornado.
NVL2(**x, valor1, valor2**)	Retorna **valor1** se **x** não é nulo; caso contrário, **valor2** é retornado.

Quadro 9 - Funções de caractere – manipulação de caracteres NVL() e NVL2()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através das funções NVL() e NVL2(), permitirá retornar valores predeterminados, caso o valor da cadeia de caracteres seja nulo ou não nulo.

- No exemplo **NVL(**F.DT_NASCIMENTO,'01/01/1900'**):** será retornado à data ‘01/01/1900’, caso o valor da cadeia de caracteres referente a data de nascimento do funcionário seja nulo.
- No exemplo **NVL2(**F.DT_NASCIMENTO,SYSDATE,'01/01/1900'**):** será retornado à data do sistema (função SYSDATE do Oracle), caso o valor da cadeia de caracteres, referente à data de nascimento do funcionário, seja não nulo, e a data ‘01/01/1900’, caso o valor da cadeia de caracteres seja nulo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXEMPLO – FUNÇÕES NVL() e NVL2()
SELECT F.DT_NASCIMENTO                    ,
       NVL(F.DT_NASCIMENTO,'01/01/1900')  ,
       NVL2(F.DT_NASCIMENTO,SYSDATE,'01/01/1900')
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 9 - Exemplo de código-fonte SQL – Funções NVL() e NVL2()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando "SELECT" com as funções NVL() e NVL2().

DT_NASCIMENTO	NVL(F.DT_NASCIMENTO,'01/01/1900')	NVL2(F.DT_NASCIMENTO,SYSDATE,'01/01/1900')
10/02/86	10/02/86	20/06/17
15/08/70	15/08/70	20/06/17
18/07/86	18/07/86	20/06/17
(null)	01/01/00	01/01/00
21/04/87	21/04/87	20/06/17
08/09/98	08/09/98	20/06/17
07/10/96	07/10/96	20/06/17

Figura 13 - Resultado do SELECT, com as funções NVL() e NVL2()
Fonte: Elaborado pela autora (2017)

1.3.9 Função SQL – Manipulação de caracteres – REPLACE()

É uma função para manipulação de caracteres, por meio da qual é possível substituir uma *string* por outra *string*. Pode ser substituída uma *string* inteira ou parte dela, dependerá do valor informado para busca dentro da cadeia de caracteres.

Função	Descrição
REPLACE(**x, string_busca, string_substituta**)	Procura "**string_busca**" em **x** e substitui por "**string_substituta**"

Quadro 10 - Funções de caractere – manipulação de caracteres REPLACE()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através da função REPLACE(), permitirá substituir uma *string* por outra *string*. No exemplo a seguir, serão substituídas todas as ocorrências

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

do caractere correspondente à letra ‘A’, pelo caractere ‘*’ (asterisco), no campo nome do funcionário.

```
-- EXEMPLO – FUNÇÃO REPLACE()
SELECT F.NM_FUNCIONARIO                    ,
       REPLACE(F.NM_FUNCIONARIO,'A','*')
FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 10 - Exemplo de código-fonte SQL – Função REPLACE()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função REPLACE().

NM_FUNCIONARIO	REPLACE(F.NM_FUNCIONARIO,’A’,’*’)
ANA MARIA	*N* M*RI*
ROSA MARIA	ROS* M*RI*
ANTONIA CAMARGO	*NTONI* C*M*RGO
JOÃO DA SILVA	JOÃO D* SILV*
JOAQUIM XAVIER	JO*QUIM X*VIER
JOSÉ MARIA	JOSÉ M*RI*
JOSÉ LISBOA	JOSÉ LISBO*

Figura 14 - Resultado do SELECT com a função REPLACE()
Fonte: Elaborado pela autora (2017)

1.3.10 Função SQL – Manipulação de caracteres – SUBSTR()

É uma função para manipulação de caracteres, por meio da qual é possível recuperar parte de uma *string*, com o comprimento predeterminado, e ainda a partir de uma posição também predeterminada.

Função	Descrição
SUBSTR(**x, inicio** [,**comprimento**])	Retorna uma substring de **x** que começa na posição especificada por início. O comprimento pode ser fornecido opcionalmente.

Quadro 11 - Funções de caractere – manipulação de caracteres SUBSTR()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Ao utilizar comando SELECT, através da função SUBSTR (), será possível recuperar parte de uma *string*.

- No exemplo **SUBSTR(**F.NM_FUNCIONARIO,3**):** será recuperada parte do conteúdo do campo nome do funcionário, a partir da terceira posição do nome do funcionário.
- No exemplo **SUBSTR(**F.NM_FUNCIONARIO,3, 5**):** será recuperada parte do conteúdo do campo nome do funcionário, a partir da terceira posição do nome do funcionário e com um comprimento de cinco caracteres.

```
    -- EXEMPLO – FUNÇÃO SUBSTR()
   SELECT F.NM_FUNCIONARIO                  ,
          SUBSTR(F.NM_FUNCIONARIO,3)        ,
          SUBSTR(F.NM_FUNCIONARIO,3, 5)
FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 11 - Exemplo de código-fonte SQL – Função SUBSTR ()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função SUBSTR ().

NM_FUNCIONARIO	SUBSTR(F.NM_FUNCIONARIO,3)	SUBSTR(F.NM_FUNCIONARIO,3,5)
ANA MARIA	A MARIA	A MAR
ROSA MARIA	SA MARIA	SA MA
ANTONIA CAMARGO	TONIA CAMARGO	TONIA
JOÃO DA SILVA	ÃO DA SILVA	ÃO DA
JOAQUIM XAVIER	AQUIM XAVIER	AQUIM
JOSÉ MARIA	SÉ MARIA	SÉ MA
JOSÉ LISBOA	SÉ LISBOA	SÉ LI

Figura 15 - Resultado do SELECT com a função SUBSTR ()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 UTILIZANDO FUNÇÕES – FUNÇÕES NUMÉRICAS

São funções utilizadas para manipulação de números:

- As funções abordadas seguem a sintaxe padronizada pela Oracle.
- As funções numéricas, normalmente, recebem um número e retornam um número. Podemos utilizá-las para realizar cálculos matemáticos em geral.

2.1 Função SQL – Funções Numéricas

São funções para manipulação de números. Abordaremos algumas delas:

- **ROUND():** permite recuperar um número arredondado, com um número determinado de casas decimais, caso informado, e em zero casas decimais, caso não for informado.
- **MOD():** permite recuperar o resto da divisão de um número.
- **TRUNC():** permite recuperar um número truncado (cortado), em um número determinado de casas decimais, caso informado, e em zero casas decimais, caso não for informado.
- **ABS():** permite recuperar o valor absoluto de número, ou seja, o número desprezando o sinal positivo ou negativo.
- **SQRT():** permite recuperar o valor da raiz quadrada de um número informado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Função	Descrição
ROUND(**x**, [**y**])	Retorna o resultado do arredondamento de **x** com **y** casas decimais (opcional). Se **y** omitido, arredonda em **0** casas decimal. Se **y** for negativo, **x** será arredondado à esquerda do ponto decimal.
MOD(**x**, **y**)	Retorna o resto, quando **x** é divido por **y**.
TRUNC(**x**, [**y**])	Retorna o resultado do truncamento de **x** com **y** com casas decimais opcionais. Se **y** omitido, arredonda em **0** casa decimal. Se **y** for negativo, **x** será arredondado à esquerda do ponto decimal.
ABS(**x**)	Retorna o valor absoluto de **x**. Ignora o sinal (**+** ou **-**).
SQRT(**x**)	Retorna a raíz quadrada de **x**.

Quadro 12 - Funções numéricas
Fonte: Elaborado pela autora (2017)

O comando SELECT, através das funções numéricas, retornará respectivamente:

- No exemplo **ROUND(**F.VL_SALARIO**):** será recuperado o valor do campo salário do funcionário arredondado, na parte inteira do número, uma vez que a quantidade de casas decimais não foi informada.
- No exemplo **ROUND(**F.VL_SALARIO,1**):** será recuperado o valor do campo salário do funcionário arredondado, a partir da primeira casa decimal.
- No exemplo **TRUNC(**F.VL_SALARIO**):** será recuperado o valor do campo salário do funcionário truncado (cortado), na parte inteira do número, uma vez que a quantidade de casas decimais não foi informada. Neste caso não há arredondamento.
- No exemplo **TRUNC(**F.VL_SALARIO,1**):** será recuperado o valor do campo salário do funcionário truncado (cortado), na primeira casa decimal. Neste caso não há arredondamento.
- No exemplo **MOD(**F.VL_SALARIO,3**):** será recuperado o valor do resultado do resto divisão do valor do campo salário do funcionário (dividendo) pelo divisor três.
- No exemplo **ABS(**F.VL_SALARIO**):** será recuperado o valor absoluto (sem sinal) do valor do campo salário do funcionário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- No exemplo **SQRT(**F.VL_SALARIO**):** será recuperado o valor da raiz quadrada referente ao valor do campo salário do funcionário.

```
 -- EXEMPLO – FUNÇÕES NUMÉRICAS
SELECT F.VL_SALARIO              ,
       ROUND(F.VL_SALARIO)       ,
       ROUND(F.VL_SALARIO,1)     ,
       TRUNC(F.VL_SALARIO)       ,
       TRUNC(F.VL_SALARIO,1)     ,
       MOD(F.VL_SALARIO,3)       ,
       ABS(F.VL_SALARIO)         ,
       SQRT(F.VL_SALARIO)
 FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 12 - Exemplo de código-fonte SQL – Funções numéricas
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções numéricas.

VL_SALARIO	ROUND(F.VL_SALARIO)	ROUND(F.VL_SALARIO,1)	TRUNC(F.VL_SALARIO)	TRUNC(F.VL_SALARIO,1)	MOD(F.VL_SALARIO,3)	ABS(F.VL_SALARIO)	SQRT(F.VL_SALARIO)
1234,56	1235	1234,6	1234	1234,5	1,56	1234,56	35,13630600959639886393338464041805575975
2345	2345	2345	2345	2345	2	2345	48,42520005121300477207350 8,5061554758954
7654,99	7655	7655	7654	7654,9	1,99	7654,99	87,49279970374705068436597081010308845903
3452,12	3452	3452,1	3452	3452,1	2,12	3452,12	58,75474448927507675473744951255150979414
980	980	980	980	980	2	980	31,30495168499705574972843136223786729616
1789	1789	1789	1789	1789	1	1789	42,29657196511320112162885521810233030698
1780	1780	1780	1780	1780	1	1780	42,19004621945797299958144258618469535728

Figura 16 - Resultado do SELECT com as funções numéricas
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 UTILIZANDO FUNÇÕES – FUNÇÕES DE CONVERSÃO

- As funções abordadas seguem a sintaxe padronizada pela Oracle.
- As funções de conversão permitem converter um valor de entrada em um outro valor de saída após a execução da função.

Podemos, por exemplo, converter um valor de um determinado tipo de dado em um valor de um outro tipo de dado.

3.1 Função SQL – Funções de conversão

São funções que permitem converter um valor de entrada para um outro valor de saída, onde:

- **TO_CHAR():** permite recuperar um valor convertido para uma *string* do tipo VARCHAR2. Se fornecido um formato, será recuperado o valor no formato informado.
- **TO_NUMBER():** permite recuperar um valor convertido em número do tipo NUMBER.
- **TO_DATE():** permite recuperar um valor convertido em uma data do tipo DATE.

A relação de formatos possíveis pode ser encontrada na referência bibliográfica:
PRICE, J. **Oracle Database 11g SQL**. São Paulo: Bookman, 2009.
Capítulo 5, tabelas 5-1 e 5-2, p. 159-163.

Função	Descrição
TO_CHAR(**x**, [, **formato**])	Converte **x** em uma string **VARCHAR2**. Pode ser fornecido um **formato** (opcional), indicando o formato de **x**.
TO_NUMBER(**x**, [, **formato**])	Converte **x** em um valor **NUMBER**.
TO_DATE(**x**, [, **formato**])	Converte **x** em um valor **DATE**.

Quadro 13 - Funções de conversão
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.2 Função SQL – Exemplo: TO_CHAR()

O comando SELECT, através das função de conversão TO_CHAR(), retornará, respectivamente:

- No exemplo **TO_CHAR(**F.VL_SALARIO, 'L9999999.99'**):** será recuperado o valor do campo salário do funcionário, convertido no padrão de símbolo de moeda local (conforme configurações de localidade).
- No exemplo **TO_CHAR(**F.VL_SALARIO, 'C99,999.99'**):** será recuperado o valor do campo salário do funcionário, convertido no padrão de símbolo de moeda ISO.
- No exemplo **TO_CHAR(**F.VL_SALARIO, '99,999.99'**):** será recuperado o valor do campo salário do funcionário, convertido no formato numérico especificado.
- No exemplo **TO_CHAR(**F.NR_MATRICULA, '0099999'**):** será recuperado o valor do campo salário do funcionário, formatado numericamente, as posições à esquerda não preenchidas pelo valor serão preenchidas com zero.

```
      -- EXEMPLO - FUNÇÕES DE CONVERSÃO - TO_CHAR()
SELECT F.VL_SALARIO ,
       TO_CHAR(F.VL_SALARIO, 'L9999999.99') "SIMBOLO MOEDA LOCAL" ,
       TO_CHAR(F.VL_SALARIO, 'C99,999.99') "SIMBOLO MOEDA ISO" ,
       TO_CHAR(F.VL_SALARIO, '99,999.99')  ,
       F.NR_MATRICULA ,
       TO_CHAR(F.NR_MATRICULA, '0099999')
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 13 - Exemplo de código-fonte SQL – Funções de conversão – TO_CHAR()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de conversão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

VL_SALARIO	SIMBOLO MOEDA LOCAL	SIMBOLO MOEDA ISO	TO_CHAR(F.VL_SALARIO,'99,999.99')	NR_MATRICULA	TO_CHAR(F.NR_MATRICULA,'0099999')
1234,56	R$1234.56	BRL1, 234.56	1,234.56	1234	0001234
2345	R$2345.00	BRL2, 345.00	2,345.00	1235	0001235
7654,99	R$7654.99	BRL7, 654.99	7,654.99	3245	0003245
3452,12	R$3452.12	BRL3, 452.12	3,452.12	2233	0002233
980	R$980.00	BRL980.00	980.00	7866	0007866
1789	R$1789.00	BRL1, 789.00	1,789.00	9876	0009876
1780	R$1780.00	BRL1, 780.00	1,780.00	9899	0009899

Figura 17 - Resultado do SELECT com as funções de conversão – TO_CHAR()
Fonte: Elaborado pela autora (2017)

3.3 Função SQL – Exemplo: TO_NUMBER()

O comando SELECT, através da função de conversão TO_NUMBER(), retornará respectivamente:

- No exemplo **TO_NUMBER(**'103'**):** será recuperado o valor informado, convertido em número.
- No exemplo **TO_NUMBER(**'103'**)** + 30**:** será recuperado o valor informado, convertido em número e somado ao valor número trinta.
- No exemplo **TO_NUMBER(**'-$12,345.99','$99,999.99'**):** será recuperado o valor informado, convertido para um número decimal, no formato de moeda.

Todos os valores informados estão no formato texto, expressados entre apóstrofos, no parâmetro esperado pela função TO_NUMBER().

```
-- EXEMPLO – FUNÇÕES DE CONVERSÃO – TO_NUMBER()
SELECT TO_NUMBER('103')        ,
       TO_NUMBER('103') + 30 ,
       TO_NUMBER('-$12,345.99','$99,999.99')
  FROM DUAL;
```

Código-fonte 14 - Exemplo de código-fonte SQL – Funções de conversão – TO_NUMBER()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função de conversão TO_NUMBER().

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

TO_NUMBER('103')	TO_NUMBER('103')+30	TO_NUMBER('-$12L345.99','$99,999.99')
103	133	-12345,99

Figura 18 - Resultado do SELECT com as funções de conversão – TO_NUMBER()
Fonte: Elaborado pela autora (2017)

3.4 Função SQL – Exemplo: TO_DATE()

O comando SELECT, através da função de conversão TO_DATE(), retornará respectivamente:

- No exemplo **TO_DATE(**F.DT_ADMISSAO,'DD/MM/YYYY'**):** será recuperado o valor do campo data de admissão para uma data no formato dia, mês e ano.

```
-- EXEMPLO – FUNÇÕES DE CONVERSÃO – TO_DATE()
SELECT F.DT_ADMISSAO ,
       TO_DATE(F.DT_ADMISSAO,'DD/MM/YYYY')
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 15 - Exemplo de código-fonte SQL – Funções de conversão – TO_DATE()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando "SELECT" com a função de conversão TO_DATE().

DT_ADMISSAO	TO_DATE(F.DT_ADMISSAO,'DD/MM/YYYY')
09/08/2010	09/08/2010
07/06/2005	07/06/2005
11/02/2003	11/02/2003
08/09/2011	08/09/2011
05/04/2010	05/04/2010
17/03/2009	17/03/2009
17/03/2009	17/03/2009

Figura 19 - Resultado do SELECT, com as funções de conversão – TO_DATE()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.5 Função SQL – Exemplo: CAST (X as TIPO)

Utilizada para converter um valor de um tipo de dados para outro tipo de dados compatível.

Conversões válidas assinaladas com X.

	Para						
De	BINARY_FLOAT BINARY_DOUBLE	CHAR VARCHAR2	NUMBER	DATE TIMESTAMP INTERVAL	RAW	ROWID UROWID	NCHAR NVARCHAR2
BINARY_FLOAT BINARY_DOUBLE	X	X	X				X
CHAR VARCHAR2	X	X	X	X	X	X	
NUMBER	X	X	X				X
DATE TIMESTAMP INTERVAL		X		X			
RAW		X			X		
ROWID UROWID		X				X	
NCHAR NVARCHAR2	X		X	X	X	X	X

Quadro 14 - Conversões válidas em um CAST()
Fonte: Price (2009)

O comando SELECT, através da função de conversão CAST(), retornará respectivamente:

- No exemplo **CAST(12345.67 AS VARCHAR2(10)):** será recuperada a *string* correspondente à conversão do valor do literal informado, do tipo de dado NUMBER(), para o tipo VARCHAR2.
- No exemplo **CAST('29/09/2013' AS DATE):** será recuperada a data correspondente à conversão do valor do literal informado, do tipo de dado VARCHAR2 para o tipo DATE.
- No exemplo **CAST(12345.678 AS NUMBER(10,2)):** será recuperado o número correspondente à conversão do valor do literal informado, do tipo de dado NUMBER(8,3) para o tipo NUMBER(10,2).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXEMPLO – FUNÇÕES DE CONVERSÃO – CAST()
-- CONVERTER VALORES LITERAIS EM TIPOS ESPECÍFICOS
SELECT
        CAST( 12345.67 AS VARCHAR2(10) ) ,
        CAST( '29/09/2013' AS DATE )     ,
        CAST( 12345.678 AS NUMBER(10,2))
  FROM DUAL;
```

Código-fonte 16 - Exemplo de código-fonte SQL – Funções de conversão – CAST()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função de conversão CAST().

CAST(12345.67ASVARCHAR2(10))	CAST('29/09/2013'ASDATE)	CAST(12345.678ASNUMBER(10,2))
12345,67	29/09/2013	12345,68

Figura 20 - Resultado do SELECT com as funções de conversão – CAST()
Fonte: Elaborado pela autora (2017)

O comando SELECT, através da função de conversão CAST(), retornará respectivamente:

- No exemplo **CAST (** F.VL_SALARIO AS VARCHAR2(10) **):** será recuperada a *string* correspondente a conversão do valor da coluna salário do funcionário do tipo de dado NUMBER() para o tipo VARCHAR2.
- No exemplo **CAST (** F.VL_SALARIO + 1000 AS NUMBER (8,2) **):** será recuperado o número correspondente a conversão da coluna salário do funcionário, somado ao literal 1000, do tipo de dado NUMBER(7,2) para o tipo NUMBER(8,2).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
 -- EXEMPLO – FUNÇÕES DE CONVERSÃO – CAST()
 -- CONVERTER VALORES DE COLUNA DE UM TIPO PARA OUTRO
SELECT
       CAST ( F.VL_SALARIO AS VARCHAR2(10) ) ,
       CAST ( F.VL_SALARIO + 1000 AS NUMBER (8,2) )
  FROM T_SIP_FUNCIONARIO F;
```

Código-fonte 17 - Exemplo de código-fonte SQL – Funções de conversão – CAST()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função de conversão CAST().

CAST(F.VL_SALARIOASVARCHAR2(10))	CAST(F.VL_SALARIO+1000ASNUMBER(8,2))
1234,56	2234,56
2345	3345
7654,99	8654,99
3452,12	4452,12
980	1980
1789	2789
1780	2780

Figura 21 - Resultado do SELECT com as funções de conversão – CAST()
Fonte: Elaborado pela autora (2017)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 UTILIZANDO FUNÇÕES – FUNÇÕES DE DATA

- As funções abordadas seguem a sintaxe padronizada pela Oracle.
- As funções de data permitem manipular data e hora no Oracle.

4.1 Função SQL – Funções de data

São algumas funções que permitem manipular datas. Seguem algumas delas:

- **SYSDATE:** permite recuperar a data do sistema.
- **ADD_MONTHS():** permite recuperar uma data, incrementada de n meses. Assim, podemos subtrair ou somar meses a uma data.
- **LASTDAY():** permite recuperar o último dia do mês da data informada.
- **MONTHS_BETWEEN():** permite recuperar o número de meses entre duas datas.
- **NEXT_DAY():** permite recuperar o próximo dia da semana, a partir da data informada.

4.2 Função SQL – Exemplo: SYSDATE

O comando SELECT, através da função de data SYSDATE, no exemplo, será recuperada a data do sistema.

```
-- EXEMPLO – SYSDATE
SELECT SYSDATE FROM DUAL;
```

Código-fonte 18 - Exemplo de código-fonte SQL – Funções de data – SYSDATE
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função de data SYSDATE.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SYSDATE
25/06/2017 12:10:44

Figura 22 - Resultado do SELECT com a função de data – SYSDATE
Fonte: Elaborado pela autora (2017)

4.3 Função SQL – Exemplo: ADD_MONTHS()

O comando SELECT, através das função de data ADD_MONTHS(), retornará respectivamente:

- No exemplo **ADD_MONTHS(SYSDATE, 1):** será recuperada a data do sistema, acrescida de um mês, ou seja, será adicionado um mês a data informada.
- No exemplo **ADD_MONTHS(SYSDATE, -1):** será recuperada a data do sistema, incrementada de um mês negativo, ou seja, será subtraído um mês da data informada.

```
      -- EXEMPLO - FUNÇÕES DE DATA - ADD_MONTHS()
SELECT SYSDATE                    "DATA ATUAL"   ,
       ADD_MONTHS(SYSDATE, 1)     "PRÓXIMO MÊS"  ,
       ADD_MONTHS(SYSDATE, -1)    "MÊS ANTERIOR"
  FROM DUAL;
```

Código-fonte 19 - Exemplo de código-fonte SQL – Funções de data – ADD_MONTHS()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de data.

DATA ATUAL	PRÓXIMO MÊS	MÊS ANTERIOR
25/06/2017 12:19:16	25/07/2017 12:19:16	25/05/2017 12:19:16

Figura 23 - Resultado do SELECT com as funções de data – ADD_MONTHS()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- No exemplo **ADD_MONTHS(DT_ADMISSAO, 1):** será recuperada a data de admissão do funcionário, incrementada de um mês, ou seja, será adicionado um mês a data informada.

```
      -- EXEMPLO – FUNÇÕES DE DATA – ADD_MONTHS()
SELECT DT_ADMISSAO ,
       ADD_MONTHS(DT_ADMISSAO,1)
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 20 - Exemplo de código-fonte SQL – Funções de data – ADD_MONTHS()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de data.

DT_ADMISSAO	ADD_MONTHS(DT_ADMISSAO,1)
09/08/2010 12:00:00	09/09/2010 12:00:00
07/06/2005 12:00:00	07/07/2005 12:00:00
11/02/2003 12:00:00	11/03/2003 12:00:00
08/09/2011 12:00:00	08/10/2011 12:00:00
05/04/2010 12:00:00	05/05/2010 12:00:00
17/03/2009 12:00:00	17/04/2009 12:00:00
17/03/2009 12:00:00	17/04/2009 12:00:00

Figura 24 - Resultado do SELECT com as funções de data – ADD_MONTHS()
Fonte: Elaborado pela autora (2017)

4.4 Função SQL – Exemplo: LAST_DAY()

O comando SELECT, através das função de data LAST_DAY(), a partir do exemplo **LAST_DAY(SYSDATE)**, será recuperada a data do último dia do mês, referente à data do sistema informada.

```
      -- EXEMPLO – FUNÇÕES DE DATA – LAST_DAY()
SELECT SYSDATE "DATA ATUAL" ,
       LAST_DAY(SYSDATE) "DATA COM ÚLTIMO DIA DO MÊS"
  FROM DUAL;
```

Código-fonte 21 - Exemplo de código-fonte SQL – Funções de data – LAST_DAY()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução do comando “SELECT” com as funções de data.

DATA ATUAL	DATA COM ÚLTIMO DIA DO MÊS
25/06/2017 12:39:37	30/06/2017 12:39:37

Figura 25 - Resultado do SELECT com as funções de data – LAST_DAY()
Fonte: Elaborado pela autora (2017)

- No exemplo **LAST_DAY(DT_ADMISSAO):** será recuperada a data do último dia do mês referente a data de admissão do funcionário.

```
      -- EXEMPLO – FUNÇÕES DE DATA – LAST_DAY()
SELECT DT_ADMISSAO ,
       LAST_DAY(DT_ADMISSAO)
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 22 - Exemplo de código-fonte SQL – Funções de data – LAST_DAY()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de data.

DT_ADMISSAO	LAST_DAY(DT_ADMISSAO)
09/08/2010 12:00:00	31/08/2010 12:00:00
07/06/2005 12:00:00	30/06/2005 12:00:00
11/02/2003 12:00:00	28/02/2003 12:00:00
08/09/2011 12:00:00	30/09/2011 12:00:00
05/04/2010 12:00:00	30/04/2010 12:00:00
17/03/2009 12:00:00	31/03/2009 12:00:00
17/03/2009 12:00:00	31/03/2009 12:00:00

Figura 26 - Resultado do SELECT com as funções de data – LAST_DAY()
Fonte: Elaborado pela autora (2017)

4.5 Função SQL – Exemplo: MONTHS_BETWEEN ()

O comando SELECT, através das função de data MONTHS_BETWEEN(), retornará, respectivamente:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- No exemplo **MONTHS_BETWEEN(**SYSDATE, DT_ADMISSAO**):** será recuperada a diferença em número de meses, entre a data do sistema e a data de admissão do funcionário.
- No exemplo **TRUNC(MONTHS_BETWEEN(**SYSDATE, DT_ADMISSAO**)) || ' MESES':** será recuperada a diferença em número de meses, entre a data do sistema e a data de admissão do funcionário. A função TRUNC truncará, ou seja, cortará o resultado obtido na parte inteira do número, o operador de concatenação "||" concatenará a palavra meses ao resultado do obtido com a execução das funções.

```
      -- EXEMPLO - FUNÇÕES DE DATA - MONTHS_BETWEEN()
SELECT DT_ADMISSAO ,
       SYSDATE     ,
       MONTHS_BETWEEN(SYSDATE, DT_ADMISSAO) ,
       TRUNC(MONTHS_BETWEEN(SYSDATE, DT_ADMISSAO)) || ' MESES'
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 23 - Exemplo de código-fonte SQL – Funções de data – MONTHS_BETWEEN()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando "SELECT" com as funções de data.

DT_ADMISSAO	SYSDATE	MONTHS_BETWEEN(SYSDATE,DT_ADMISSAO)	TRUNC(MONTHS_BETWEEN(SYSDATE,DT_ADMISSAO))||'MESES'
09/08/2010	25/06/2017	82,5333673088410991636798088410991636798 1	82 MESES
07/06/2005	25/06/2017	144,5978834378733572281959378733572281 96	144 MESES
11/02/2003	25/06/2017	172,6488511798088410991636798088410991 64	172 MESES
08/09/2011	25/06/2017	69,5757253733572281959378733572281959 3787	69 MESES
05/04/2010	25/06/2017	86,6623995669056152927120669056152927 1207	86 MESES
17/03/2009	25/06/2017	99,2753027927120669056152927170669056 1529	99 MESES
17/03/2009	25/06/2017	99,2753027927120669056152927120669056 1529	99 MESES

Figura 27 - Resultado do SELECT com as funções de data – MONTHS_BETWEEN()
Fonte: Elaborado pela autora (2017)

4.6 Função SQL – Exemplo: NEXT_DAY ()

O comando SELECT, através das função de data NEXT_DAY(), retornará conforme exemplo **NEXT_DAY(**SYSDATE, 1**)**, a próxima data referente ao dia da semana informado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Os valores de 1 a 7 representam, respectivamente, os dias da semana de domingo a sábado.

```
      -- EXEMPLO – FUNÇÕES DE DATA – NEXT_DAY()
SELECT SYSDATE ,
       NEXT_DAY(SYSDATE, 1) "PRÓXIMO DOMINGO" ,
       NEXT_DAY(SYSDATE, 2) "PRÓXIMA SEGUNDA" ,
       NEXT_DAY(SYSDATE, 3) "PRÓXIMA TERÇA"   ,
       NEXT_DAY(SYSDATE, 4) "PRÓXIMA QUARTA"  ,
       NEXT_DAY(SYSDATE, 5) "PRÓXIMA QUINTA"  ,
       NEXT_DAY(SYSDATE, 6) "PRÓXIMA SEXTA"   ,
       NEXT_DAY(SYSDATE, 7) "PRÓXIMO SÁBADO"
  FROM DUAL;
```

Código-fonte 24 - Exemplo de código-fonte SQL – Funções de data – NEXT_DAY()
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de data.

SYSDATE	PRÓXIMO DOMINGO	PRÓXIMA SEGUNDA	PRÓXIMA TERÇA	PRÓXIMA QUARTA	PRÓXIMA QUINTA	PRÓXIMA SEXTA	PRÓXIMO SÁBADO
25/06/2017	02/07/2017	26/06/2017	27/06/2017	28/06/2017	29/06/2017	30/06/2017	01/07/2017

Figura 28 - Resultado do SELECT com as funções de data – NEXT_DAY()
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 EXEMPLOS ADICIONAIS ENVOLVENDO DATAS

5.1 Função SQL – Exemplo: TO_CHAR ()

Com o comando SELECT, através da função TO_CHAR(), será recuperado respectivamente:

- No exemplo **TO_CHAR(**DT_ADMISSAO, 'YYYY'**):** será recuperado o ano data de admissão do funcionário.
- No exemplo **TO_CHAR(**DT_ADMISSAO, 'MM/YYYY'**):** será recuperado o mês e ano da data de admissão do funcionário.
- No exemplo **TO_CHAR(**DT_ADMISSAO, 'DD/MM'**):** será recuperado o dia e o mês da data de admissão do funcionário.
- No exemplo **TO_CHAR(**DT_ADMISSAO, 'DD " de " **fmMONTH** " de " YYYY', **'NLS_DATE_LANGUAGE=PORTUGUESE'):** será recuperada a data por extenso referente à data de admissão do funcionário.

Onde fmMONTH é um modelo de formato disponível na linguagem SQL – ORACLE, e NLS_DATE_FORMAT, permite especificar o formato de data padrão, para ser usado com as funções TO_CHAR() e TO_DATE().

```
     -- EXEMPLO – FUNÇÕES DE DATA
SELECT DT_ADMISSAO ,
       TO_CHAR(DT_ADMISSAO, 'YYYY')      "ANO CONTRATACAO"        ,
       TO_CHAR(DT_ADMISSAO, 'MM/YYYY') "MÊS E ANO CONTRATACAO"   ,
       TO_CHAR(DT_ADMISSAO, 'DD/MM') "DIA E MÊS DA CONTRATACAO" ,
       TO_CHAR(DT_ADMISSAO, 'DD " de " fmMONTH " de " YYYY',
'NLS_DATE_LANGUAGE=PORTUGUESE') "DATA POR EXTENSO"
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 25 - Exemplo de código-fonte SQL – Funções de data
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de data.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

DT_ADMISSAO	ANO CONTRATACAO	MÊS E ANO CONTRATACAO	DIA E MÊS DA CONTRATACAO	DATA POR EXTENSO
09/08/2010	2010	08/2010	09/08	09 de AGOSTO de 2010
07/06/2005	2005	06/2005	07/06	07 de JUNHO de 2005
11/02/2003	2003	02/2003	11/02	11 de FEVEREIRO de 2003
08/09/2011	2011	09/2011	08/09	08 de SETEMBRO de 2011
05/04/2010	2010	04/2010	05/04	05 de ABRIL de 2010
17/03/2009	2009	03/2009	17/03	17 de MARÇO de 2009
17/03/2009	2009	03/2009	17/03	17 de MARÇO de 2009

Figura 29 - Resultado do SELECT com as funções de data
Fonte: Elaborado pela autora (2017)

5.2 Função SQL – Exemplo: TO_DATE ()

Com o comando SELECT, através da função TO_DATE(), será recuperado respectivamente:

- No exemplo **TO_DATE(**DT_ADMISSAO , 'DD/MM/YYYY'**) – 5:** será recuperada a data de admissão do funcionário, subtraída de 5 dias.
- No exemplo **TO_DATE(**SYSDATE, 'DD/MM/YYYY'**) – TO_DATE(**DT_ADMISSAO, 'DD/MM/YYYY'**):** será recuperada a diferença entre as duas datas, o resultado é apresentado em número de dias.

```
      -- EXEMPLO – FUNÇÕES DE DATA
SELECT DT_ADMISSAO ,
       TO_DATE(DT_ADMISSAO , 'DD/MM/YYYY') - 5,
       TO_DATE(SYSDATE, 'DD/MM/YYYY') –
                TO_DATE(DT_ADMISSAO, 'DD/MM/YYYY')
 FROM  T_SIP_FUNCIONARIO;
```

Código-fonte 26 - Exemplo de código-fonte SQL – Funções de data
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de data.

DT_ADMISSAO	TO_DATE(DT_ADMISSAO,'DD/MM/YYYY')-5	TO_DATE(SYSDATE,'DD/MM/YYYY')-TO_DATE(DT_ADMISSAO,'DD/MM/YYYY')
09/08/2010	04/08/2010	2512
07/06/2005	02/06/2005	4401
11/02/2003	06/02/2003	5248
08/09/2011	03/09/2011	2117
05/04/2010	31/03/2010	2638
17/03/2009	12/03/2009	3022
17/03/2009	12/03/2009	3022

Figura 30 - Resultado do SELECT com as funções de data
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Atenção: para executar os comandos anteriores, altere a formatação de saída para:

ALTER SESSION SET NLS_DATE_FORMAT ='DD/MM/YYYY';

- No exemplo **TO_DATE(**DT_ADMISSAO,'DD/MM/YYYY HH:MI:SS'**) + 6/24:** será recuperada a data de admissão do funcionário, somadas 6 horas. O 6/24 representa 6 horas das 24 horas do dia.

```
      -- EXEMPLO – FUNÇÕES DE DATA
SELECT DT_ADMISSAO ,
       TO_DATE(DT_ADMISSAO,'DD/MM/YYYY HH:MI:SS') + 6/24
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 27 - Exemplo de código-fonte SQL – Funções de data
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com as funções de data.

DT_ADMISSAO	TO_DATE(DT_ADMISSAO,'DD/MM/YYYYHH:MI:SS')+6/24
09/08/2010 12:00:00	09/08/2010 06:00:00
07/06/2005 12:00:00	07/06/2005 06:00:00
11/02/2003 12:00:00	11/02/2003 06:00:00
08/09/2011 12:00:00	08/09/2011 06:00:00
05/04/2010 12:00:00	05/04/2010 06:00:00
17/03/2009 12:00:00	17/03/2009 06:00:00
17/03/2009 12:00:00	17/03/2009 06:00:00

Figura 31 - Resultado do SELECT com as funções de data
Fonte: Elaborado pela autora (2017)

Atenção: para executar os comandos anteriores, altere a formatação de saída para:
ALTER SESSION SET NLS_DATE_FORMAT ='DD/MM/YYYY HH:MI:SS';

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

DATABASE SQL REFERENCE, **ORACLE**. Disponível em: <https://docs.oracle.com/cd/B19306_01/server.102/b14200/sql_elements004.htm>. Acesso em: 28 jan. 2025.

ELMASRI, R.; NAVATHE, S.B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

HEUSER, C.A. **Projeto de Banco de Dados. Série Livros Didáticos.** v. 4. 6. ed. Rio Grande do Sul: Bookman, 2009.

MACHADO, Felipe Nery R. **Banco de Dados - Projeto e Implementação**. 2. ed. São Paulo: Érica, 2008.

PRICE, J. **Oracle Database 11g SQL**. São Paulo: Bookman, 2009.

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados: Implementação em SQL, PL/SQL e Oracle 11g.** São Paulo: Pearson Education do Brasil, 2013.

SILBERSCHATZ, A; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

DRS	Data Retrieve Language (Linguagem de recuperação de dados).
DQL	Data Query Language (Linguagem para consulta de dados).
SELECT	Comando da linguagem SQL responsável por permitir a consulta ou recuperação de registros ou linhas nas tabelas.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com